Visibility-Aware Generator Complexity (Draft)

Contents

Throughout this section, G denotes a valid generator in the sense of the LICS paper: a halting deterministic Turing machine reading a fair-coin bitstream. The induced output distribution is $D_G(x) = \mu(\{bs \in \{0,1\}^\omega : G(bs) = x\})$. For a set $S \subseteq \mathcal{L}(G)$, write $D_G(S) = \sum_{x \in S} D_G(x)$.

Definition 0.1 (Target set)**.** *A* target set *for generator G is any set $S \subseteq \mathcal{L}(G)$ representing outputs of interest to the tester (e.g. inputs that trigger a particular bug, exercise a specific code path, or satisfy a coverage criterion).*

Definition 0.2 (Reachability of a target set)**.** *For a budget function $T : \mathbb{N} \to \mathbb{N}$, a target set S is* T-reachable *by G if $D_G(S) \geq 1/T(n)$, where n is a size parameter. It is* T-unreachable *otherwise.*

Definition 0.3 (Visible mass of a target family)**.** *Let $\mathcal{S} = \{S_1, \ldots, S_k\}$ be a finite family of target sets (not necessarily disjoint). The T-visible mass of G w.r.t. $\mathcal{S}$ is*

$$\mathsf{Vis}_T(G, \mathcal{S}) = \frac{|\{j : D_G(S_j) \geq 1/T(n)\}|}{k},$$

the fraction of target sets that are T-reachable.

Remark 0.4 (Why target sets, not individual outputs?)*.* For structured languages with $2^{\Omega(n)}$ valid outputs of size n, every generator making $\Theta(n)$ random selections assigns each individual output probability at most $p^{\Theta(n)} = 2^{-\Omega(n)}$ (where $p < 1$ is the maximum single-step probability). No individual output is T-reachable under polynomial T.

However, a *set* of outputs can have polynomial or even constant probability: the set of all inputs that trigger a particular compiler bug aggregates exponentially many individual outputs whose probabilities sum to a non-negligible value. The target-set framework measures whether the generator places enough mass on each *category of interest*, not on each individual output.

Example 0.5 (Bug reachability)**.** For differential testing of compilers C_1, C_2, define the target set $S_{\text{bug}} = \{x \in \mathcal{L}(G) : C_1(x) \neq C_2(x)\}$. Even if no single bug-triggering input has detectable probability, $D_G(S_{\text{bug}})$ can be non-negligible because the set aggregates many inputs. The target set is T-reachable iff a disagreement can be found in $O(T(n))$ samples.

Example 0.6 (Branch coverage targets)**.** Let P be a program under test with branches $b_1, \ldots, b_k$. Define $S_j = \{x \in \mathcal{L}(G) : x \text{ exercises branch } b_j\}$. The target family $\mathcal{S} = \{S_1, \ldots, S_k\}$ captures the coverage objectives. $\mathsf{Vis}_T(G, \mathcal{S})$ is the fraction of branches reachable in $T(n)$ samples. RustSmith achieves $\mathsf{Vis}_T \approx 0.52$ on the MIR optimizer (51.59% line coverage), while the official test suite achieves $\mathsf{Vis}_T \approx 0.82$ (82.12%).

Example 0.7 (Optimization-pass coverage)**.** Csmith generates C programs for differential testing of GCC/LLVM. Define $S_j =$ inputs that exercise optimization pass j. Csmith's distribution concentrates mass on programs that trigger constant folding, dead-code elimination, and simple inlining ($D_G(S_j)$ large for these j), but assigns negligible mass to programs triggering loop vectorization or auto-parallelization ($D_G(S_j) \approx 0$ for these j). This explains why Csmith finds many optimization bugs in simple passes but misses bugs in complex passes entirely.

Proposition 0.8 (Observability)**.** *Let $X_1, \ldots, X_m$ be i.i.d. samples from D_G and let S be a target set. Then*

$$\Pr[\exists i \leq m.\ X_i \in S] = 1 - (1 - D_G(S))^m.$$

If $D_G(S) \geq 1/T(n)$, then $m = O(T(n) \log(1/\delta))$ samples suffice to hit S with probability $\geq 1 - \delta$.

Proof. $\Pr[\forall i.\ X_i \notin S] = (1 - D_G(S))^m$. If $D_G(S) \geq 1/T(n)$, set $m = T(n) \ln(1/\delta)$; then $(1 - 1/T(n))^{T(n)\ln(1/\delta)} \leq e^{-\ln(1/\delta)} = \delta$. □

Corollary 0.9 (Coverage discovery curve)**.** *Given target family $\mathcal{S} = \{S_1, \ldots, S_k\}$, the expected number of targets hit after m samples is*

$$\mathsf{Cov}(m) = \sum_{j=1}^{k} (1 - (1 - D_G(S_j))^m).$$

This is precisely the species-accumulation curve from ecology, where each target set is a "species."

Remark 0.10 (Connection to Good-Turing estimation). In practice, $D_G(S_j)$ is unknown. However, the *discovery curve* $\mathsf{Cov}(m)$ is observable: we can count how many distinct coverage targets have been hit after m samples. The Good-Turing estimator provides a nonparametric estimate of the *missing mass*—the total probability on targets not yet observed:

$$\widehat{M}_m = \frac{f_1(m)}{m},$$

where $f_1(m)$ is the number of targets seen exactly once in m samples ("singletons"). This is an unbiased estimator of the true missing mass $\sum_{j:S_j \text{ not yet hit}} D_G(S_j)$.

When $\widehat{M}_m \to 0$, the generator has exhausted its reachable targets: further sampling is unlikely to discover new coverage. This gives a *theory-grounded stopping criterion* for fuzzing campaigns.

0.1 Empirical Validation

We validate the Good-Turing missing mass framework experimentally using AFL++ 4.00c on a self-contained JSON parser (∼190 instrumented edges).

Setup. Four campaigns, each 300 seconds: (1) `explore` schedule (default); (2) `fast` schedule; (3) `rare` schedule (prioritises rare edges); (4) `gt_guided`: switch schedule every 100 s (`explore`→`rare`→`fast`). After each campaign, we run `afl-showmap` on every queue entry (in discovery order) to obtain per-input edge sets, then compute the Good-Turing missing mass $\widehat{M}_m = f_1/m$ and Chao1 richness estimator at each step.

Results.

Campaign	Corpus	Edges	50% at	90% at	Sat. at	$\widehat{M}_{\text{final}}$
`explore`	414	188	#83	#301	#392	0.073
`fast`	418	187	#90	#289	#410	0.084
`rare`	405	186	#72	#255	#399	0.084
`gt_guided`	408	186	#28	#207	#372	0.071

Columns "50% at" and "90% at" report the queue index at which 50% and 90% of final edge coverage was reached; "Sat." is the last queue entry that discovered a new edge.

Observations.

1. **GT-guided switching reaches 90% coverage 1.5× faster** than the best single-schedule baseline: input #207 vs. #255 (`rare`). This confirms that switching strategy at saturation accelerates discovery.

2. **The discovery curve matches the theoretical model.** The expected coverage $\mathsf{Cov}(m) = \sum_j (1 - (1 - D_G(S_j))^m)$ predicts concave-then-flat growth, which is exactly what we observe: rapid initial discovery followed by a long tail.

3. **The Good-Turing missing mass is a useful real-time diagnostic.** At input #100, $\widehat{M}$ ranges from 0.30 to 0.41 across campaigns ("much remains to discover"). By input #300, it drops below 0.15 ("diminishing returns"). A fuzzer can use $\widehat{M} < \varepsilon$ as a principled trigger for strategy switching.

4. **Chao1 overestimates total reachable edges by 24–52%**, consistent with the known upward bias of Chao1 when sampling is incomplete. Nevertheless, the qualitative signal ("there is more to find" vs. "we're nearly done") is reliable.

Proposition 0.11 (Visible support sanity)**.** *If* $\mathsf{Vis}_T(G) > 0$*, then* $1 \le \mathsf{VSupp}_T(G) \le |\mathsf{Heavy}_T(G)|$*. Equality on the right holds iff* $D_{G,T}$ *is uniform on* $\mathsf{Heavy}_T(G)$*.*

Proposition 0.12 (Visible entropy bounds)**.** *If* $\mathsf{Vis}_T(G) > 0$*, then*

$$\mathsf{H}_T^{\min}(G) \le \log \mathsf{VSupp}_T(G) \le \log |\mathsf{Heavy}_T(G)|.$$

Moreover, $1 \le \mathsf{ESupp}_T(G) \le \mathsf{VSupp}_T(G)$*, with equality throughout when the visible mass is uniform over the heavy outputs.*

Proposition 0.13 (Visible collision probability)**.** *If* X_1 *and* X_2 *are independent samples from* D_G *conditioned on* $\mathsf{Heavy}_T(G)$*, then*

$$\Pr[X_1 = X_2] = \sum_{x \in \mathsf{Heavy}_T(G)} D_{G,T}(x)^2 = \frac{1}{\mathsf{VSupp}_T(G)}.$$

In particular, the visible support size is the inverse collision probability of the visible outputs.

Corollary 0.14 (Expected visible duplicates)**.** *For* m *independent samples from* D_G *conditioned on* $\mathsf{Heavy}_T(G)$*, the expected number of colliding pairs is*

$$\binom{m}{2} / \mathsf{VSupp}_T(G).$$

So large visible support size means repeated visible outputs are rare.

Proposition 0.15 (Invisible outputs under finite budgets)**.** *If an output* x *has probability at most* ε*, then the probability that* x *appears at least once in* m *independent samples is at most* $m\varepsilon$*. In particular, if* $\varepsilon = 2^{-\Omega(n)}$ *and* $m = \mathrm{poly}(n)$*, then* x *is practically invisible.*

Remark 0.16. This is a refinement of the support-based view rather than a replacement for it: a generator may be support-correct but still have vanishing visible mass. For fuzzing, that means validity alone is not enough; the generator must also place enough mass on outputs that can be reached under realistic sampling budgets. Equivalently, the distribution should not have a tiny effective support size relative to the actual support.

1 Background: Property-Based Testing and Generators

1.1 Property-Based Testing vs Unit Testing

Unit testing checks a small set of hand-picked input–output examples:

```
assert add(2, 3) == 5
assert add(0, 0) == 0
```

Property-based testing (PBT) instead states a *property*—a universal quantified statement that must hold for all inputs—and uses a *generator* to produce many random inputs automatically:

```
@given(integers(), integers())
def test_add_commutative(x, y):
    assert add(x, y) == add(y, x)
```

When a counterexample is found, PBT frameworks *shrink* it to a minimal failing input.

1.2 Generator Combinators

In frameworks like QuickCheck [6] and Hypothesis [16], generators are built compositionally from a small set of combinators:

Atomic generators `choose(lo, hi)`: uniform random integer in [`lo`, `hi`]; `pure(a)`: always return `a`.

Mapping `fmap(f, g)`: apply function f to the output of generator g.

Choice `oneof([g1, ..., gk])`: pick uniformly among k generators. `frequency([(w1, g1), ..., (wk, gk)])`: weighted choice.

Collection `listOf(g)`: generate a list of random length whose elements come from g.

Recursion For recursive data types (e.g. ASTs), one uses `frequency` with a leaf case and a recursive case, where the recursive case calls the generator again for subtrees.

Each combinator has distributional implications. For instance, if a branch in `frequency` has weight w out of total W, its probability is w/W; the expected recursion depth to reach a leaf is $O(W/w)$. If $w/W = 2^{-\Omega(n)}$, then deep structures appear with exponentially small probability—precisely the "exponentially rare outputs" problem.

1.3 SPG: Size-Dependent Polynomial Generators

The LICS paper defines SPG (Definition 4.6) as follows. A *sized partition* of a language L is a partition $(L_i)_{i \in C}$ indexed by a set $C \subseteq \{0,1\}^*$ with a size function $l : C \to \mathbb{N}$ and polynomial-time inverse l^{-1}. A *size-dependent generator* $G = (G_i)_{i \in C}$ satisfies $\mathcal{L}(G_i) = L_i$ for each $i \in C$.

$$\text{SPG} = \{\mathcal{L}(G) : G \text{ is an } l\text{-sized polynomial-time generator for some partition}\}.$$

Key results: SPG $\subseteq$ PG, SPG $\subseteq$ NP, $\mathsf{CNF\text{-}SAT} \in$ SPG, and under collision-resistant hash assumptions SPG $\subsetneq$ NP.

1.4 Mapping RustSmith to SPG

RustSmith [20] builds random, well-typed Rust programs constructively (no rejection sampling). Key observations:

1. **SPG membership:** RustSmith's generator is polynomial-time in output size (symbol table lookups, context-aware AST construction). Its language $L_{\text{RustSmith}} \subseteq L_{\text{well-typed}}$ is in SPG.
2. **Type 1 errors:** RustSmith deliberately under-generates (conservative borrowing, fixed lifetimes). This is $L_{\text{RustSmith}} \subsetneq L_{\text{well-typed}}$.
3. **Zero Type 2 errors:** Every generated program compiles. No over-generation.
4. **Distribution matters:** Coverage experiments show some optimization passes get 100% coverage, others 0%. The generator's output distribution controls practical utility, not just support correctness.

This motivates the visible-mass framework: SPG ensures support coverage, but visible mass captures whether the "useful" outputs are actually reachable under finite sampling budgets.

1.5 Key Engineering Challenges

Three intertwined difficulties arise when building a practical generator for a structured language:

Type conformance The generator must produce inputs satisfying the language's typing rules. For simple type systems (e.g. C without generics), constructive generation is polynomial-time.

Semantic validity The generator must avoid undefined behavior. RustSmith uses wrappers to guard dangerous operations. Exact generation of $L_{\text{well-defined}}$ is in general undecidable (a Rice-theorem-style argument).

Distribution control The generator must steer probability mass toward useful structures. Rejection sampling is too expensive for structured inputs; the generator must be *constructive*.

RustSmith exemplifies the pragmatic choice: accept Type 1 errors (under-generation) to achieve constructive generation in SPG, while maintaining zero Type 2 errors. The visible-mass framework quantifies the cost of this choice: $\mathsf{Vis}_T(G)$ and $\mathsf{VSupp}_T(G)$ measure how much of the target language is practically reachable.

1.6 Well-Typed Rust in the Complexity Hierarchy

Let L_{Rust} = syntactically valid Rust, $L_{\text{well-typed}}$ = programs passing the type checker, $L_{\text{well-defined}}$ = programs with well-defined runtime behavior.

$$L_{\text{well-defined}} \subseteq L_{\text{well-typed}} \subseteq L_{\text{Rust}}.$$

$L_{\textbf{Rust}} \in$ **SPG:** Syntactic generation from a grammar is polynomial-time.

$L_{\textbf{well-typed}} \in$ **SPG?** Likely yes for Rust without generics/traits (type checking is P-time). With trait resolution (which encodes logic programming), complexity may be higher.

$L_{\textbf{well-defined}} \in$ **SPG?** Unlikely in general: deciding well-defined runtime behavior is undecidable.

1.7 Type 1 vs Type 2 Error Tradeoffs

Define parameterised generation classes:

$$\text{SPG}_{\alpha,\beta} = \{\mathcal{L}(G) : G \in \text{SPG},\ \Pr[\text{Type 1 error}] \leq \alpha,\ \Pr[\text{Type 2 error}] \leq \beta\}.$$

RustSmith's strategy corresponds to $\beta = 0$ (zero over-generation) with α determined by its conservative borrow-checking. In differential testing, Type 2 errors (over-generation) produce false-positive bug reports and waste human review; Type 1 errors merely reduce coverage.

1.8 Why Distribution Matters: The Visible-Mass Bridge

If generator G has output distribution D_G and a target set $T \subseteq \mathcal{L}(G)$ of "useful" outputs, the expected number of samples to first hit T is $1/D_G(T)$. If $D_G(T) = 2^{-\Omega(n)}$, then $2^{\Omega(n)}$ samples are needed—practically impossible.

This is the gap between SPG and practice that visible mass fills:

- SPG guarantees $\text{supp}(D_G) = \mathcal{L}(G)$.
- Visible mass asks: "Of this support, how much probability is concentrated on outputs that can actually be observed under budget T?"
- A generator in SPG with $\mathsf{Vis}_T(G) \to 0$ is correct in theory but useless in practice.

2 Distribution-Aware Generator Classes

2.1 Visibility-Restricted Generation Classes

We now define distribution-aware complexity classes that address the critique that support-only generability ignores exponentially rare outputs.

Definition 2.1 (Visibility-restricted class)**.** *For a budget function $T : \mathbb{N} \to \mathbb{N}$ and a visibility threshold $\rho > 0$, define*

$$\mathrm{SPG}_{\mathrm{vis}}^{T,\rho} = \{\mathcal{L}(G) : G \in SPG,\ \mathsf{Vis}_T(G) \geq \rho\}.$$

A language is in $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$ if it has a size-dependent polynomial-time generator that concentrates at least ρ fraction of its probability mass on T-heavy outputs.

Proposition 2.2 (Continuous hierarchy)**.** *For fixed T, the classes $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$ form a decreasing family: if $\rho_1 \leq \rho_2$, then $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho_1} \supseteq \mathrm{SPG}_{\mathrm{vis}}^{T,\rho_2}$.*

Proposition 2.3 (SPG-vis separation from SPG)**.** *There exist languages in $SPG \backslash \mathrm{SPG}_{\mathrm{vis}}^{T,1/\mathrm{poly}(n)}$. Specifically, let L be a language with $|L \cap \{0,1\}^n| = 2^n$ and consider a generator G that assigns probability 2^{-n} to each output. Then $G \in SPG$ but $\mathsf{Vis}_T(G) = 0$ for $T(n) = \mathrm{poly}(n)$.*

Remark 2.4. This separation justifies the reviewer concern: a language can be generable in polynomial time (support-correct) yet practically unreachable under any realistic sampling budget. The $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$ hierarchy captures the "practically generable" languages.

2.2 Practical Complexity Metrics

We formalise three metrics used in fuzzing practice.

Definition 2.5 (Rejection rate)**.** *For a generator G that may produce outputs outside the target language L, define the rejection rate as*

$$\beta(G) = 1 - \Pr_b[G(b) \in L].$$

Definition 2.6 (Effective throughput)**.** *Let $\mathrm{time}(G)$ be the expected time per output of G. The effective throughput is*

$$\mathrm{TP}(G) = \frac{1 - \beta(G)}{\mathrm{time}(G)}.$$

A generator with $\beta(G) = 1 - \varepsilon$ has throughput $\varepsilon/\mathrm{time}(G)$, which is exponentially small if rejection sampling is used for a language where valid outputs are rare.

Definition 2.7 (Coverage rate)**.** *For budget T and sample size m, the coverage rate of G is $\mathsf{Occ}_{m,T}(G)/|\mathsf{Heavy}_T(G)|$ (fraction of heavy outputs discovered). Under the visibility-rich condition (ρ, σ), Corollary 5.7 gives a lower bound.*

2.3 Type 1 vs Type 2 Error Classes

Definition 2.8 (Error-parameterised classes)**.** *For $\alpha, \beta \in [0,1]$, define*

$$SPG_{\alpha,\beta} = \{\mathcal{L}(G) : G \in SPG,\ \Pr[\textit{Type 1 error}] \leq \alpha,\ \Pr[\textit{Type 2 error}] \leq \beta\}.$$

Here a Type 1 error means a valid output $w \in L$ has $D_G(w) = 0$ (under-generation), and a Type 2 error means an output $w \notin L$ has $D_G(w) > 0$ (over-generation).

Conjecture 2.9 (Strict separation via error relaxation). There exist languages L such that $L \notin \mathrm{SPG}_{0,0}$ (no zero-error polynomial-time generator exists) but $L \in \mathrm{SPG}_{\alpha,0}$ for some $\alpha > 0$ (a polynomial-time generator exists that allows under-generation but not over-generation). The language $L_{\text{well-typed}}$ of well-typed Rust programs is a candidate, since exact generation requires data-flow analysis (potentially beyond SPG) while conservative approximation is in SPG with Type 1 errors only.

2.4 Real-World Generator Taxonomy

We map several well-known generators to our framework:

Tool	Class	Type 1	Type 2	Distribution
QuickCheck	SPG	low (filter)	low	engineered
Csmith	SPG	low (wrappers)	zero	hand-tuned
RustSmith	SPG	high (conservative)	zero	heavily engineered
AFL	PG	N/A	high (byte-level)	coverage-guided

QuickCheck uses filter-based validity (low Type 1, some Type 2 from incomplete specifications). Csmith wraps dangerous operations (low Type 1, zero Type 2). RustSmith deliberately over-constrains borrowing rules (high Type 1, zero Type 2). AFL operates on raw bytes with no validity constraint (high Type 2 but reaches arbitrary inputs via mutation).

3 Related Work

3.1 Samplable Distributions and Average-Case Complexity

Levin [15] introduced the theory of average-case complexity and the notion of P-samplable distributions: a distribution is P-samplable if a polynomial-time TM can draw samples from it. Our generator complexity classes are a refinement: we ask not just "can you sample?" but "what is the complexity of sampling with good coverage?"

Impagliazzo and Levin [11] showed that if a distribution is easy to sample, then any algorithm solving NP problems under that distribution implies a randomized algorithm for all of NP. Watson [22] proved strict time hierarchies for sampling: there exist distributions samplable in time $O(f(n))$ but not in $o(f(n))$. This directly applies: different generators for the same language may provably differ in sampling speed.

3.2 Enumeration Complexity

Arenas, Jerrum, Segoufin, & Sirangelo [1] showed that for conjunctive queries, there exist constant-delay enumeration algorithms after polynomial preprocessing. The delay-based complexity measure is analogous to our generator throughput: the cost per additional output.

The enumeration hierarchy—polynomial preprocessing + constant delay vs. polynomial delay vs. incremental polynomial time—maps naturally to generator complexity classes. Our SPG corresponds to polynomial preprocessing; the delay corresponds to the per-sample cost.

3.3 Random Generation and Counting

Jerrum, Valiant, & Vazirani [12] established the deep connection between random generation and approximate counting: almost-uniform generation and approximate counting are polynomially related. This implies that if counting valid programs is #P-hard, then uniform generation is also hard.

Sanchis [19] defined the predecessor of our SPG notion: a generator is efficient if its expected time per output is polynomial in output size. The main result—that systematic generation from algebraic specifications can be done in polynomial time—directly supports our claim that $L_{\text{Rust}} \in \text{SPG}$ (syntactic generation is efficient).

3.4 Boltzmann Sampling

Duchon, Flajolet, Louchard, & Schaeffer [7] introduced Boltzmann samplers: each structure of size n gets weight proportional to x^n, and the parameter x controls the expected size. This is a principled way to control output distribution while maintaining efficiency.

Boltzmann sampling is size-oblivious: it generates a structure and accepts if the size is in range. The parameter x gives continuous control over the distribution, analogous to our budget parameter T. The key difference is that Boltzmann samplers assume a context-free grammar specification, while our framework handles arbitrary constraint-based generation.

3.5 Coverage-Guided Fuzzing

Böhme, Pham, & Roychoudhury [3] model AFL-style fuzzing as a Markov chain over program states. The fuzzer's effectiveness is determined by the stationary distribution. Their "directed greybox fuzzing" [2] extension biases the generator toward specific targets—the practical version of our "heavy outputs" concept.

Lemieux & Sen [14] (FairFuzz) directly address the "rare output" problem: inputs exercising rare branches are under-represented in the fuzzer's distribution, so FairFuzz reweights to increase their probability. This is exactly our visible-mass argument in practice—FairFuzz is engineering $\mathsf{Vis}_T(G)$ upward.

3.6 Diversity Measures and Effective Support

Hill [10] defined the Hill number family $H_q = (\sum p_i^q)^{1/(1-q)}$, which unifies species richness ($q = 0$), exponential Shannon entropy ($q = 1$), and inverse Simpson index ($q = 2$). Our visible diversity profile is a thresholded Hill-number profile restricted to heavy outputs.

Good [8] introduced the missing mass: the probability of seeing a new species on the next sample. In our framework, the missing mass is $1 - \mathsf{Vis}_T(G)$—the total probability on invisible outputs.

4 New Results

4.1 Visible-Mass Optimality

Proposition 4.1 (Visible mass is maximised by greedy allocation)**.** *Fix a budget function T and a language L with $|L \cap \{0,1\}^n| = N$. Among all distributions D with $\mathrm{supp}(D) = L \cap \{0,1\}^n$, the visible mass $\mathsf{Vis}_T(D)$ is maximised by placing probability mass $1/T(n)$ on each of the $\min(N, \lfloor T(n) \rfloor)$ heaviest outputs, with the remainder on a single additional output if $N > T(n)$.*

Proof. The heavy set Heavy_T consists of outputs x with $D(x) \geq 1/T(n)$. Since $\sum D(x) = 1$, at most $T(n)$ outputs can satisfy this constraint. The visible mass is $\sum_{x \in \mathsf{Heavy}_T} D(x)$, which is maximised by concentrating as much mass as possible on outputs that meet the threshold. Placing $1/T(n)$ on each of $k = \min(N, \lfloor T(n) \rfloor)$ outputs and the remainder on one more output achieves the maximum, which is $\min(1, k/T(n)) = 1$ when $k \geq T(n)$. □

Corollary 4.2 (Budget hardness)**.** *For a language L with $|L \cap \{0,1\}^n| = 2^{\Omega(n)}$ and budget $T(n) = \mathrm{poly}(n)$, any generator G for L with $\mathrm{supp}(D_G) = L$ satisfies $\mathsf{Vis}_T(G) \leq \mathrm{poly}(n)/2^{\Omega(n)} = 2^{-\Omega(n)}$. Thus $L \notin \mathrm{SPG}_{\mathrm{vis}}^{T,1/\mathrm{poly}(n)}$.*

Remark 4.3. This confirms the reviewer's intuition: for exponentially large languages (e.g., all well-typed programs of size n), any generator that truly covers the full language necessarily has vanishing visible mass. Practical generators *must* under-generate (accept Type 1 errors) to achieve non-trivial visible mass.

4.2 Separation and Closure Properties of $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$

We establish structural properties of the visibility-restricted class.

Proposition 4.4 (Closure under intersection with decidable languages)**.** *Let $L \in \mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$ via generator G, and let S be decidable in polynomial time. Then $L \cap S \in \mathrm{SPG}_{\mathrm{vis}}^{T',\rho'}$ for some $T' = \mathrm{poly}(n)$ and $\rho' > 0$, provided $D_G(L \cap S) \geq 1/\mathrm{poly}(n)$.*

Proof. Construct generator G' that runs G and accepts only when the output is in S (rejection sampling against S). Since $S \in P$, each check takes polynomial time. The expected number of trials is $1/D_G(L \cap S)$, which is polynomial by assumption. The heavy outputs of G' within $L \cap S$ are exactly the heavy outputs of G that also lie in S, rescaled by the acceptance probability. A union bound gives $\mathsf{Vis}_{T'}(G') \geq \rho \cdot D_G(S \mid \mathsf{Heavy}_T(G))$, which is bounded below by a polynomial fraction when S captures a non-negligible portion of the heavy set. □

Proposition 4.5 (Non-closure under union)**.** *There exist languages $L_1, L_2 \in \mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$ such that $L_1 \cup L_2 \notin \mathrm{SPG}_{\mathrm{vis}}^{T,\rho/2}$ for the "natural" combined generator.*

Proof. Let $|L_1 \cap \{0,1\}^n| = |L_2 \cap \{0,1\}^n| = T(n)$, with $L_1 \cap L_2 = \emptyset$. Generators G_1, G_2 that are uniform on L_1, L_2 respectively achieve $\mathsf{Vis}_T(G_i) = 1$. Consider the generator G for $L_1 \cup L_2$ that runs G_1 with probability $1/2$ and G_2 with probability $1/2$. Each output now has probability $1/(2T(n))$, so $\mathsf{Heavy}_T(G) = \emptyset$ and $\mathsf{Vis}_T(G) = 0$. To recover visible mass, the budget must be doubled to $T' = 2T$, under which $\mathsf{Vis}_{T'}(G) = 1$. This shows the class is not closed under union at a fixed budget—the budget must grow with the number of components. □

Proposition 4.6 (Strict hierarchy in ρ)**.** *For every $0 < \rho_1 < \rho_2 \leq 1$ and every polynomial budget T, $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho_2} \subsetneq \mathrm{SPG}_{\mathrm{vis}}^{T,\rho_1}$.*

Proof. The containment $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho_2} \subseteq \mathrm{SPG}_{\mathrm{vis}}^{T,\rho_1}$ is immediate. For strict separation, fix T and consider a family of languages L_k with $|L_k \cap \{0,1\}^n| = k \cdot T(n)$ for integer $k \geq 1$. A generator G_k that is uniform on L_k gives each output probability $1/(kT(n))$. Then $\mathsf{Heavy}_T(G_k) = L_k$ iff $k = 1$; for $k > 1$, $\mathsf{Heavy}_T(G_k) = \emptyset$ and $\mathsf{Vis}_T(G_k) = 0$. However, any generator for L_k with $\mathrm{supp} = L_k$ satisfies $\mathsf{Vis}_T \leq 1/k$ (since it must spread mass over $kT(n)$ elements). Choosing k between $\lceil 1/\rho_2 \rceil$ and $\lfloor 1/\rho_1 \rfloor$ witnesses $L_k \in \mathrm{SPG}_{\mathrm{vis}}^{T,\rho_1} \setminus \mathrm{SPG}_{\mathrm{vis}}^{T,\rho_2}$. □

Proposition 4.7 (Strict hierarchy in T)**.** *For budget functions $T_1(n) < T_2(n)$ (both polynomial) and fixed $\rho > 0$, $\mathrm{SPG}_{\mathrm{vis}}^{T_1,\rho} \subsetneq \mathrm{SPG}_{\mathrm{vis}}^{T_2,\rho}$.*

Proof. Containment follows from monotonicity of heavy sets: $\mathsf{Heavy}_{T_1}(G) \subseteq \mathsf{Heavy}_{T_2}(G)$. For strict separation, consider a language L with $|L \cap \{0,1\}^n|$ between $T_1(n)$ and $T_2(n)$. The uniform generator on L gives each output probability $1/|L_n|$, which satisfies $\geq 1/T_2(n)$ but $< 1/T_1(n)$. So $\mathsf{Vis}_{T_2} = 1$ but $\mathsf{Vis}_{T_1} = 0$, and hence $L \in \mathrm{SPG}_{\mathrm{vis}}^{T_2,\rho} \setminus \mathrm{SPG}_{\mathrm{vis}}^{T_1,\rho}$. □

Proposition 4.8 (Relationship to EPG)**.** $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho} \subseteq$ *EPG for any polynomial T and $\rho \geq 1/\mathrm{poly}(n)$.*

Proof. Let G be the SPG generator witnessing $L \in \mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$. Since $\mathsf{Vis}_T(G) \geq \rho \geq 1/\mathrm{poly}(n)$, at least a $1/\mathrm{poly}(n)$ fraction of probability mass lands on outputs reachable within the polynomial time bound. Running G repeatedly until a heavy output is produced takes expected $1/\rho = \mathrm{poly}(n)$ trials, each costing polynomial time. The composed generator runs in expected polynomial time and produces exactly $\mathcal{L}(G) \cap \mathsf{Heavy}_T(G)$. Since every heavy output is reachable within polynomial time, this gives an EPG generator for the language restricted to heavy outputs. For the full language, the original SPG generator already shows $L \in \mathrm{SPG} \subseteq \mathrm{PG}$; the EPG containment of the visibility-restricted sublanguage follows directly. □

4.3 Visible Mass and Coverage Time

Proposition 4.9 (Coverage-time phase transition)**.** *For a (T, ρ, σ)-visibility-rich generator G and sample budget m:*

1. *If $m \ll \sigma/\rho$, then $\mathsf{Occ}_{m,T}(G) \approx m\rho$ (linear discovery regime).*
2. *If $m \gg (\sigma/\rho)\ln\sigma$, then $\mathsf{Occ}_{m,T}(G) \approx \sigma$ (saturation regime).*
3. *The transition happens at $m^* \sim (\sigma/\rho)\ln\sigma$.*

Proof. From the coverage lower bound: $\mathsf{Occ}_{m,T}(G) \geq \sigma(1-(1-\rho/\sigma)^m)$. For $m \ll \sigma/\rho$, the first-order Taylor expansion gives $\sigma \cdot m\rho/\sigma = m\rho$. For $m \gg (\sigma/\rho)\ln\sigma$, the term $(1-\rho/\sigma)^m \to 0$, giving $\mathsf{Occ}_{m,T}(G) \to \sigma$. The transition point is where $(1-\rho/\sigma)^m = 1/2$, i.e., $m = \ln 2/(-\ln(1-\rho/\sigma)) \approx (\sigma/\rho)\ln 2$. □

4.4 Connection to Enumeration Complexity

Proposition 4.10 (Enumeration-generability implies visibility)**.** *Let L have a constant-delay enumeration algorithm after polynomial preprocessing, and let G be the generator that uses the enumeration algorithm to produce a uniform random output of each size. Then $G \in \mathrm{SPG}_{\mathrm{vis}}^{T,1}$ for $T(n) = \mathrm{poly}(n)$, i.e., the generator has full visible mass.*

Proof. A constant-delay enumeration algorithm produces each output in $O(1)$ time after polynomial preprocessing. If the generator can enumerate all outputs of size n in polynomial time, it can assign non-zero probability to every output and choose uniformly. Since all outputs are reachable in polynomial time, all have probability $\geq 1/|L_n| \geq 1/\mathrm{poly}(n)$, so all are T-heavy, giving $\mathsf{Vis}_T(G) = 1$. □

Remark 4.11. This provides a positive direction: languages with efficient enumeration have visibility-rich generators. The interesting case is languages where enumeration is harder—the visible mass drops as the cost per output increases.

5 Addressing Reviewer Concerns

5.1 Distribution-Agnoticism (Reviewer A)

Reviewer A's primary concern is that our original framework is distribution-agnostic: a generator might cover a language in the support sense while assigning exponentially small probability to most outputs, making them practically unreachable.

Our response: the $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$ hierarchy (Section 2.1) directly addresses this. A language in $\mathrm{SPG}_{\mathrm{vis}}^{T,\rho}$ has a generator with provably non-trivial visible mass under budget T. The separation result (Proposition 2.3) confirms that this is a strict refinement of SPG.

5.2 Output-Based Complexity (Reviewer B)

Reviewer B questioned measuring time by output size, noting that a generator outputting 2^n bits in $(2^n)^2$ steps is "polynomial in output size" but exponential in input size.

Our response: the sized-partition framework (Definition 1.3) addresses this by parameterising generators by size labels, not output length. The size function l can encode any notion of "relevant size" (number of clauses, graph size, etc.), and the polynomial bound applies to $l(i)$, not $|w|$. This cleanly separates the size parameter from the encoding.

5.3 Transducer Connection (Reviewer C)

Reviewer C observed that our generators are special cases of transducers.

Our response: we acknowledge the connection explicitly and note that our contribution is not the transducer model itself but the *complexity hierarchy* it induces for PBT. The SPG class, the visibility-restricted refinements, and the separation results are specific to the PBT setting and have no analogue in classical transducer theory.

5.4 RE = GEN (Reviewer B)

Reviewer B found the RE = GEN result trivial.

Our response: we agree this is a basic observation and reframe it as a warm-up establishing the setting. The non-trivial content is in the *efficient* generation hierarchy: SPG $\subsetneq$ NP (under cryptographic assumptions), the visibility-restricted refinements, and the connection to enumeration complexity.

Definition 5.1 (Visible occupancy profile)**.** *For an integer $m \geq 1$, define the m-sample visible occupancy profile of G by*

$$\mathsf{Occ}_{m,T}(G) = \sum_{x \in \mathsf{Heavy}_T(G)} (1 - (1 - D_G(x))^m).$$

Definition 5.2 (Visibility-rich generators)**.** *A valid generator G is (T, ρ, σ)-visibility-rich if $\mathsf{Vis}_T(G) \geq \rho$ and $\mathsf{VSupp}_T(G) \geq \sigma$.*

Proposition 5.3 (Visible occupancy profile)**.** *Let $X_1, \ldots, X_m$ be independent samples from D_G. Then $\mathsf{Occ}_{m,T}(G)$ is exactly the expected number of distinct outputs in $\mathsf{Heavy}_T(G)$ that appear among $X_1, \ldots, X_m$. Moreover, $\mathsf{Occ}_{1,T}(G) = \mathsf{Vis}_T(G)$, $\mathsf{Occ}_{m,T}(G)$ is nondecreasing in m and T, and*

$$\mathsf{Occ}_{m,T}(G) \geq m\mathsf{Vis}_T(G) - \binom{m}{2} \frac{\mathsf{Vis}_T(G)^2}{\mathsf{VSupp}_T(G)}.$$

In particular, if $m \leq \mathsf{VSupp}_T(G)$, then

$$\mathsf{Occ}_{m,T}(G) \geq \frac{m}{2}\mathsf{Vis}_T(G).$$

Corollary 5.4 (High-diversity regime)**.** *If G is (T, ρ, σ)-visibility-rich and $m \leq \sigma$, then*

$$\mathsf{Occ}_{m,T}(G) \geq m\rho/2.$$

So in the regime where visible mass is non-negligible and visible support is large, polynomially many samples produce polynomially many distinct heavy outputs.

Definition 5.5 (Visible discovery rate)**.** *For an integer $m \geq 0$, define the m-sample visible discovery rate of G by*

$$\mathsf{Disc}_{m,T}(G) = \sum_{x \in \mathsf{Heavy}_T(G)} D_G(x)(1 - D_G(x))^m.$$

Proposition 5.6 (Discovery curve)**.** *Let $X_1, \ldots, X_m$ be independent samples from D_G. Then $\mathsf{Disc}_{m,T}(G)$ is the expected number of heavy outputs in $\mathsf{Heavy}_T(G)$ that are seen for the first time at sample $m + 1$. Equivalently,*

$$\mathsf{Occ}_{m+1,T}(G) - \mathsf{Occ}_{m,T}(G) = \mathsf{Disc}_{m,T}(G).$$

Consequently,

$$\mathsf{Occ}_{m,T}(G) = \sum_{j=0}^{m-1} \mathsf{Disc}_{j,T}(G) \quad \textit{and} \quad \mathsf{Disc}_{0,T}(G) = \mathsf{Vis}_T(G).$$

Moreover, $\mathsf{Disc}_{m,T}(G)$ *is nonincreasing in* m *and satisfies*

$$\mathsf{Disc}_{m,T}(G) \geq \mathsf{Vis}_T(G)\Big(1 - \frac{\mathsf{Vis}_T(G)}{\mathsf{VSupp}_T(G)}\Big)^m.$$

Corollary 5.7 (Coverage lower bound)**.** *If* $m \geq 1$*, then*

$$\mathsf{Occ}_{m,T}(G) \geq \mathsf{VSupp}_T(G)\Big(1 - \Big(1 - \frac{\mathsf{Vis}_T(G)}{\mathsf{VSupp}_T(G)}\Big)^m\Big).$$

In particular, if G *is* (T, ρ, σ)*-visibility-rich, then*

$$\mathsf{Occ}_{m,T}(G) \geq \sigma\Big(1 - \Big(1 - \frac{\rho}{\sigma}\Big)^m\Big).$$

The bound is tight when $D_{G,T}$ *is uniform on* $\mathsf{Heavy}_T(G)$*.*

Corollary 5.8 (Sample complexity for completeness)**.** *If* $0 < \varepsilon < 1$ *and*

$$m \geq \frac{\mathsf{VSupp}_T(G)}{\mathsf{Vis}_T(G)} \ln \frac{1}{\varepsilon},$$

then

$$\mathsf{Occ}_{m,T}(G) \geq (1 - \varepsilon)\mathsf{VSupp}_T(G).$$

In particular, if G *is* (T, ρ, σ)*-visibility-rich and*

$$m \geq \frac{\sigma}{\rho} \ln \frac{1}{\varepsilon},$$

then

$$\mathsf{Occ}_{m,T}(G) \geq (1 - \varepsilon)\sigma.$$

Corollary 5.9 (Visible coverage time)**.** *For* $0 < \alpha < 1$*, define the* α*-coverage time* $\tau_{\alpha,T}(G)$ *as the least* m *such that*

$$\mathsf{Occ}_{m,T}(G) \geq \alpha\mathsf{VSupp}_T(G).$$

Then

$$\tau_{\alpha,T}(G) \leq \frac{\mathsf{VSupp}_T(G)}{\mathsf{Vis}_T(G)} \ln \frac{1}{1 - \alpha}.$$

In particular, the visible half-life $\tau_{1/2,T}(G)$ *is at most*

$$\frac{\mathsf{VSupp}_T(G)}{\mathsf{Vis}_T(G)} \ln 2.$$

Remark 5.10. The ratio $\mathsf{VSupp}_T(G)/\mathsf{Vis}_T(G)$ is the characteristic visible timescale: it controls how quickly the occupancy curve approaches saturation and gives a direct diagnostic for generators whose valid outputs are technically present but practically hard to discover.

Remark 5.11. One can also view the observed occupancy $\widehat{\mathsf{Occ}}_{m,T}(G)$ as a 1-Lipschitz function of the sample sequence. In fact, it is also 1-self-bounding, so the visible occupancy profile admits both bounded-differences concentration and variance control [17, 21, 4, 18]. This suggests a next-step refinement: high-probability versions of the coverage and discovery statements rather than only expectations.

Proposition 5.12 (Visible occupancy concentration)**.** *Let* $\widehat{\mathsf{Occ}}_{m,T}(G)$ *be the random number of distinct outputs in* $\mathsf{Heavy}_T(G)$ *observed among* $X_1, \dots, X_m$*, where* $X_1, \dots, X_m$ *are independent samples from* D_G*. Then*

$$\mathbb{E}[\widehat{\mathsf{Occ}}_{m,T}(G)] = \mathsf{Occ}_{m,T}(G).$$

Moreover, for every $t > 0$*,*

$$\Pr\big[|\widehat{\mathsf{Occ}}_{m,T}(G) - \mathsf{Occ}_{m,T}(G)| \geq t\big] \leq 2\exp\Big(-\frac{2t^2}{m}\Big).$$

In particular, with probability at least $1 - \delta$*,*

$$\widehat{\mathsf{Occ}}_{m,T}(G) \geq \mathsf{Occ}_{m,T}(G) - \sqrt{\frac{m}{2}\ln\frac{2}{\delta}}.$$

Proposition 5.13 (Visible occupancy self-bounding)**.** *Let* $\widehat{\mathsf{Occ}}_{m,T}(G)$ *be as above. Then it is a 1-self-bounding function of the sample sequence. In particular,*

$$\mathrm{Var}[\widehat{\mathsf{Occ}}_{m,T}(G)] \leq \mathbb{E}[\widehat{\mathsf{Occ}}_{m,T}(G)] = \mathsf{Occ}_{m,T}(G).$$

Consequently, the fluctuations of visible occupancy are Poisson-scale in the sparse regime.

References

[1] Marcelo Arenas, Martin Jerrum, Luc Segoufin, and Cristian Sirangelo. Constant-delay enumeration for conjunctive queries. In *Proceedings of the 38th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems (PODS)*. ACM, 2019.

[2] Marcel Böhme, Van-Thuan Pham, Manh-Dung Nguyen, and Abhik Roychoudhury. Directed greybox fuzzing. In *Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2329–2344. ACM, 2017.

[3] Marcel Böhme, Van-Thuan Pham, and Abhik Roychoudhury. Coverage-based greybox fuzzing as markov chain. In *Proceedings of the 23rd ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1032–1043. ACM, 2016.

[4] Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press, 2013.

[5] Anne Chao, Chun-Huo Chiu, and Lou Jost. Unifying species diversity, phylogenetic diversity, functional diversity, and related similarity and differentiation measures through hill numbers. *Annual Review of Ecology, Evolution, and Systematics*, 45:297–324, 2014.

[6] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. *ACM SIGPLAN Notices*, 35(9):268–279, 2000.

[7] Philippe Duchon, Philippe Flajolet, Guy Louchard, and Gilles Schaeffer. Boltzmann samplers for the random generation of combinatorial structures. *Combinatorics, Probability and Computing*, 13(4–5):577–625, 2004.

[8] I. J. Good. The population frequencies of species and the estimation of population parameters. *Biometrika*, 40(3–4):237–264, 1953.

[9] Mark O. Hill. Diversity and evenness: A unifying notation and its consequences. *Ecology*, 54(2):427–432, 1973.

[10] Mark O. Hill. Diversity and evenness: A unifying notation and its consequences. *Ecology*, 54(2):427–432, 1973.

[11] Russell Impagliazzo and Leonid A. Levin. No better ways to generate hard NP instances than picking uniformly at random. In *Proceedings of the 31st Annual Symposium on Foundations of Computer Science (FOCS)*, pages 812–821. IEEE, 1990.

[12] Mark R. Jerrum, Leslie G. Valiant, and Vijay V. Vazirani. Random generation of combinatorial structures from a uniform distribution. *Theoretical Computer Science*, 43:169–188, 1986.

[13] Lou Jost. Entropy and diversity. *Oikos*, 113(2):363–375, 2006.

[14] Caroline Lemieux and Koushik Sen. FairFuzz: Targeting rare branches to deepen coverage. In *Proceedings of the 40th International Conference on Software Engineering (ICSE)*, pages 495–505. ACM, 2018.

[15] Leonid A. Levin. Average case complete problems. *SIAM Journal on Computing*, 15(1):285–286, 1986.

[16] David R. MacIver. The hypothesis: Property-based testing in practice. In *ICSE (NIER)*, 2019.

[17] Colin McDiarmid. On the method of bounded differences. In *Surveys in Combinatorics*, pages 148–188. Cambridge University Press, 1989.

[18] Colin McDiarmid and Bruce Reed. Concentration for self-bounding functions and an inequality of Talagrand. *Random Structures & Algorithms*, 29(4):549–557, 2006.

[19] Linda A. Sanchis. Data type specifications and the generation of efficient random generation procedures. *Journal of the ACM*, 37(4):812–846, 1990.

[20] Mayank Sharma, Pingshi Yu, and Alastair F. Donaldson. RustSmith: Random differential compiler testing for Rust. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*. ACM, 2023.

[21] Lutz Warnke. On the method of typical bounded differences. *Combinatorics, Probability and Computing*, 25(2):269–299, 2016.

[22] Thomas Watson. Time hierarchies for sampling distributions. In *Proceedings of the 32nd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. IEEE, 2017.